

Challenge Lab — Unit Converter



Challenge Overview

Difficulty	★★★☆☆ — Intermediate
Estimated Time	30–45 minutes
Skills Tested	Variables, arithmetic operators, type conversion, string formatting
Prerequisites	Labs 2.1, 2.2, 2.3 complete

Background & Context

Unit conversion is one of the most common real-world programming tasks. Engineers convert sensor readings (meters to feet), logistics software converts weights (kg to lbs), GPS systems convert speeds (km/h to mph), and scientific tools convert temperatures (Kelvin to Celsius). Writing a clear, well-labelled conversion report is a practical skill used in data engineering, automation scripting, and API output formatting.

In this challenge, you will build a **complete unit conversion script** from scratch that handles three types of measurement — distance, weight, and speed — and formats the results into a professional-looking Conversion Report.

Requirements

Build a Python script that:

1. **Defines a distance variable** in miles and converts it to kilometres
 - Conversion factor: `1 mile = 1.60934 km`
2. **Defines a weight variable** in pounds and converts it to kilograms
 - Conversion factor: `1 lb = 0.453592 kg`
3. **Defines a speed variable** in miles per hour (mph) and converts it to km/h
 - Conversion factor: `1 mph = 1.60934 km/h` (*Hint: this uses the same factor as distance*)
4. **Prints a formatted Conversion Report** that shows:
 - A clear title/header
 - Each original value with its unit
 - Each converted value with its unit

- Clean alignment and labelling

Sample Values to Use

```
distance_miles = 26.2      # Marathon distance
weight_pounds = 185.0      # Body weight
speed_mph = 60.0           # Highway speed
```

Sample Output Format

```
=====
                        UNIT CONVERSION REPORT
=====
DISTANCE:
  26.2 miles   ->   42.164708 km

WEIGHT:
  185.0 lbs    ->   83.91452 kg

SPEED:
  60.0 mph     ->   96.5604 km/h

=====
```

(Your exact decimal values and formatting style may differ — focus on correctness and readability.)

Planning Strategy

Before writing a single line of code, plan your approach:

Step 1 — List your variables What inputs do you need? What outputs will you compute? Write them out as comments first.

Step 2 — Write the calculations Each conversion is a single multiplication. Write all three calculations before moving to output.

Step 3 — Plan your output format Sketch the report on paper or in comments. Decide on column widths and separators.

Step 4 — Build the print statements Write each print line, converting numbers to strings as needed.

Step 5 — Test Run the script. Check each result against a manual calculation or an online converter.

Hints

Read these only if you are stuck — not before attempting the problem.

Hint 1 — Calculation structure Each conversion is a multiplication: `converted = original * conversion_factor`. Define your conversion factors as named variables (e.g., `MILES_TO_KM = 1.60934`) at the top of the script. This is good practice and makes the code self-documenting.

Hint 2 — Formatting the report You can create separator lines using string repetition: `"=" * 44` produces a line of 44 equals signs.

Hint 3 — Combining values and units in output You have two options:

```
# Option A – str() + concatenation
print(" " + str(distance_miles) + " miles -> " + str(distance_km) + " km")

# Option B – f-string (cleaner)
print(f" {distance_miles} miles -> {distance_km} km")
```

Hint 4 — Rounding (optional) Python's `round()` function can limit decimal places: `round(42.164708, 2)` → `42.16`. Use it if you want cleaner output, but it is not required.

Common Pitfalls

- **Multiplying instead of dividing (or vice versa):** To convert FROM miles TO km, you multiply by `1.60934`. To go from km back to miles, you divide. Make sure your direction is right.
 - **Mixing up the conversion factors:** Weight and distance/speed use different factors. Define each as a named constant so you do not accidentally swap them.
 - **Forgetting `str()` for string concatenation:** If you use `+` to build strings, every number must be wrapped in `str()`. A `TypeError` here means you forgot to convert.
 - **Hardcoding numbers twice:** Define conversion factors once at the top; do not type `1.60934` in multiple places. If the value needs to change, you only edit one line.
-

Bonus Challenges

Bonus 1 — Reverse Conversion

Extend the report to also show the **reverse conversion** for each measurement (`km → miles`, `kg → lbs`, `km/h → mph`). Add a second section to the report labelled `REVERSE CONVERSIONS`.

Hint: Division reverses multiplication. If `km = miles * 1.60934`, then `miles = km / 1.60934`.

Bonus 2 — Formatted Decimal Places

Modify your output so every converted value is displayed to **exactly 2 decimal places**. Research the `round()` function or f-string format specifier `:.2f`.

Example: 42.164708 should display as 42.16.

Bonus 3 — Additional Unit

Add a fourth conversion: temperature in Celsius to Fahrenheit (reuse the formula from Problem 1). Include it in the report with an appropriate label.

Bonus 4 — Named Constants Convention

In Python, variables that never change are conventionally named in ALL_CAPS. Rename all your conversion factors using this convention (e.g., MILES_TO_KM, LBS_TO_KG) and explain in a comment why this convention exists.

Self-Check Questions

Before submitting or moving on, verify:

- ☐ Does the script run without any errors?
 - ☐ Does the distance conversion match an online calculator for 26.2 miles?
 - ☐ Does the weight conversion match an online calculator for 185 lbs?
 - ☐ Is the report clearly labelled with original and converted units?
 - ☐ Are conversion factors stored in named variables (not typed directly into calculations)?
-

Cross References

- **Concept files:** [Lab_2_1_Variables_and_DataTypes.md], [Lab_2_2_Operators.md], [Lab_2_3_StringOps_TypeConversion.md]
- **Practice:** [Practice_Problems.md] — Problem 1 (temperature formula uses the same pattern)
- **Solutions:** [Solutions.md] — Review Problem 3 (paycheck) for formatted output inspiration
- **Quick reference:** [Revision_Sheet.md]